全连接神经网络

数据科学与大数据技术专业

第 4 讲

1 神经网络概览

1. 生物神经元与人工神经元

   - 生物神经元可以粗略抽象为兴奋与抑制两类状态. 它们通过突触接收来自其他神经元的信号, 对输入进行整合, 再决定是否向下游传递响应.
   - 人工神经元则是对这一过程的数学抽象. 给定输入向量 $x = (x_1, \ldots, x_n)^\top$, 权重向量 $w = (w_1, \ldots, w_n)^\top$ 和偏置 b, 神经元先计算

     $$z = w^\top x + b,$$

     再通过激活函数 f 得到输出 $a = f(z)$.
   - 当 f 取阶跃函数时, 这一模型对应经典的感知器; 当 f 取 Sigmoid 函数时, 它可以看作 logistic 回归的基本单元. 因此, 单个人工神经元可以视为感知器或广义线性分类器的统一抽象.

2. 神经网络的结构与功能

   - 人工神经网络由大量神经元及其之间的有向连接构成. 若多个神经元按层堆叠, 且信息仅从输入层依次流向输出层而不存在反馈连接, 就得到前馈神经网络.
   - 神经元的核心在于激活规则, 即输入到输出之间的映射关系. 激活函数通常选为非线性函数, 因为多层线性变换的复合仍然是线性变换; 若各层都采用线性映射, 则深层网络在表达能力上并不优于单层模型.
   - 网络的拓扑结构决定不同神经元之间的连接方式, 学习算法则利用训练数据不断调整权重和偏置, 从而使网络能够拟合具体任务中的输入输出关系.

3. 连接主义与符号主义

   - 神经网络早期被视为连接主义的重要代表. 20 世纪 80 年代后期, 分布式并行处理模型强调以下观点:
     - 信息表示是分布式的, 即信息通常由多个神经元共同承载, 而不是局部存储在单个单元中.
     - 记忆和知识主要编码在神经元之间的连接强度上.
     - 学习过程体现为连接强度的逐步调整.

- 误差反向传播算法的引入显著提升了神经网络的训练能力, 使其在分类、回归以及更复杂的机器学习任务中得到广泛应用.
- 与之相对, 符号主义模型强调基于显式符号和规则的推理, 更适合知识表示与逻辑决策任务; 连接主义更擅长处理复杂、非结构化数据, 但通常面临可解释性较弱的问题.

4. **问题思考**

- 神经网络在模拟生物神经系统过程中存在哪些优势和不足?
- 如何在人工神经元模型中引入更多生物神经元的特性?

2 前馈神经网络

1. 定义与结构

- 前馈神经网络是一类按层组织的神经网络, 信息从输入层依次流向隐藏层和输出层, 网络中不存在反馈连接, 因而可用有向无环图来描述.
- 常见的前馈神经网络通常采用全连接结构, 即层内神经元之间没有连接, 相邻两层之间的神经元两两相连, 这一类模型也常称为全连接神经网络或多层感知器 (MLP).

2. 以前向传播为例理解两层网络

- 以下考虑一个含单隐藏层的两层前馈神经网络. 记输入为 $\boldsymbol{x}$, 则其前向传播过程可写为

$$
\begin{aligned}
\boldsymbol{a}^{(0)} &= \boldsymbol{x}, \\
\boldsymbol{z}^{(1)} &= \boldsymbol{W}^{(1)}\boldsymbol{a}^{(0)} + \boldsymbol{b}^{(1)}, \\
\boldsymbol{a}^{(1)} &= f_1\left(\boldsymbol{z}^{(1)}\right), \\
\boldsymbol{z}^{(2)} &= \boldsymbol{W}^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)}, \\
\hat{\boldsymbol{y}} = \boldsymbol{a}^{(2)} &= f_2\left(\boldsymbol{z}^{(2)}\right).
\end{aligned}
$$

- 其中, 第 1 层负责将输入映射到隐藏表示, 第 2 层再将隐藏表示映射为最终输出. 随着层数增加, 网络可以通过多次线性变换与非线性激活的组合来提升表达能力.

3. 一般的层间传播公式

- 若将输入层记为第 0 层, 则对任意第 l 层 $(1 \le l \le L)$, 前向传播都可以写为

$$
\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}, \quad \boldsymbol{a}^{(l)} = f_l\left(\boldsymbol{z}^{(l)}\right).
$$

- 这表明每一层都先进行仿射变换, 再施加非线性激活函数. 因此也可以把输出写成递推形式

$$
\boldsymbol{a}^{(l)} = f_l\left(\boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}\right).
$$

- 常用记号如下:
  - L: 除输入层外的网络层数.
  - M_l: 第 l 层神经元的个数.
  - $f_l(\cdot)$: 第 l 层的激活函数.

- $\boldsymbol{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$: 第 $l-1$ 层到第 l 层的权重矩阵.
- $\boldsymbol{b}^{(l)} \in \mathbb{R}^{M_l}$: 第 l 层的偏置向量.
- $\boldsymbol{z}^{(l)} \in \mathbb{R}^{M_l}$: 第 l 层的净输入, 也称净活性值.
- $\boldsymbol{a}^{(l)} \in \mathbb{R}^{M_l}$: 第 l 层的输出, 也称活性值.

4. 表达能力与通用近似

   - 通用近似定理可以作如下直观理解: 在适当条件下, 具有单隐藏层的前馈神经网络只要隐藏层足够宽, 就能够在紧集上以任意精度逼近连续函数.
   - 这一结论说明神经网络可以作为强大的非线性函数逼近器, 用于学习复杂的特征变换、构造分类器, 以及刻画更复杂的输入输出关系或条件分布.
   - 需要注意的是, 通用近似定理讨论的是"存在性"而非"可训练性": 它说明合适的参数组合存在, 但并不意味着网络一定容易训练, 也不意味着较小规模的网络就足以高效完成逼近.

5. 梯度计算

   - 训练前馈神经网络的关键在于利用链式法则计算各层参数的梯度, 并据此更新权重与偏置. 下一小节将以矩阵形式说明这一过程.

6. **问题思考**

   - 两层神经网络在复杂任务上存在哪些局限性?
   - 增加隐藏层后, 对梯度计算和模型训练会带来哪些挑战?

2.1 梯度计算与权重更新

以前向传播中的第 l 层为例, 记

$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}, \quad \boldsymbol{a}^{(l)} = f_l\left(\boldsymbol{z}^{(l)}\right).$$

若单个样本对应的损失函数为 $L(\hat{y}, y)$, 则更新权重矩阵 $\boldsymbol{W}^{(l)}$ 的关键在于计算 $\frac{\partial L}{\partial \boldsymbol{W}^{(l)}}$. 定义当前层的误差项

$$\delta^{(l)} = \frac{\partial L}{\partial \boldsymbol{z}^{(l)}},$$

根据链式法则,

$$\frac{\partial L(\hat{y}, y)}{\partial \boldsymbol{W}^{(l)}} = \frac{\partial L}{\partial \boldsymbol{z}^{(l)}} \frac{\partial \boldsymbol{z}^{(l)}}{\partial \boldsymbol{W}^{(l)}} = \delta^{(l)} \left(\boldsymbol{a}^{(l-1)}\right)^{\top}.$$

对应到矩阵元素上,

$$\left[\frac{\partial L(\hat{y}, y)}{\partial \boldsymbol{W}^{(l)}}\right]_{i,j} = \delta_i^{(l)} a_j^{(l-1)}.$$

同理, 偏置项的梯度为

$$\frac{\partial L(\hat{y}, y)}{\partial \boldsymbol{b}^{(l)}} = \delta^{(l)}.$$

这说明第 l 层参数更新所需的信息非常直接: 一部分来自上一层的激活值 $\boldsymbol{a}^{(l-1)}$, 另一部分来自当前层的误差项 $\delta^{(l)}$. 训练时只要从输出层开始逐层向前计算这些误差项, 就能得到各层参数的梯度, 这正是反向传播算法的核心思想.

3 激活函数

- 选用原则:
  - 激活函数应提供足够的非线性, 并尽量做到连续或几乎处处可导, 以便利用基于梯度的数值优化方法学习网络参数.
  - 函数本身及其导数的形式应尽可能简单, 从而降低前向传播与反向传播的计算开销.
  - 导函数的取值不宜过大或过小, 否则容易造成梯度爆炸或梯度消失, 影响训练效率与稳定性.
  - 神经元的线性部分负责聚合信息, 激活函数 $a = f(z)$ 则将净输入 z 映射为输出状态. 因此, 单个神经元应尽量保持结构简单, 而网络的表达能力主要来自多层连接的组合.
  - 从经验上看, 激活值最好能够适度反映净输入的变化. 单调递增函数通常更便于分析与优化, 但在现代网络中也存在一些有效的非单调激活函数.
- 常见激活函数类型:
  - **Sigmoid 与 Tanh**:
    * 两者分别定义为

$$\sigma(x) = \frac{1}{1+\exp(-x)},$$
$$\tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)} = 2\sigma(2x) - 1.$$

    * Sigmoid 与 Tanh 都属于 S 型饱和函数. 当输入绝对值较大时, 导数会接近 0, 因而可能带来梯度消失问题.
    * Tanh 的输出范围为 $(-1, 1)$, 具有零中心化特性; Sigmoid 的输出范围为 $(0, 1)$, 输出恒为正, 这会给参数优化带来额外困难.
  - **Ramp 系列**:
    * ReLU 在正半轴保持线性、在负半轴输出 0, 形式简单且计算高效; 由于正半轴不饱和, 它在一定程度上缓解了梯度消失问题.
    * ReLU 的单侧响应形式具有一定的生物启发意义, 但也可能导致部分神经元长期落在负半轴, 从而出现"死亡 ReLU"现象.
    * Leaky ReLU 和 ELU 等函数通过在负半轴保留较小斜率或引入平滑负值输出, 试图减轻这一问题, 代价是函数形式相对更复杂.
  - **复合激活函数**:
    * Swish 函数定义为

$$\mathrm{swish}(x) = x\sigma(\beta x).$$

      其中 $\sigma(\beta x)$ 可以理解为一个依赖输入的门控因子. 当 β 增大时, 该门控更接近阶跃函数, Swish 也更接近 ReLU; 当 $\beta = 0$ 时, 它退化为线性缩放 $x/2$.
    * GELU 函数通常定义为

$$\mathrm{GELU}(x) = x\Phi(x),$$

      其中 $\Phi(x) = P(X \le x)$ 是标准高斯随机变量 $X \sim N(0, 1)$ 的分布函数. 它也可以看作一种输入相关的平滑门控机制.

* 由于 $\Phi(x)$ 没有初等闭式表达, GELU 在实现中常使用近似式:

$$\mathrm{GELU}(x) \approx 0.5x\left(1+\tanh\left(\sqrt{\frac{2}{\pi}}\left(x+0.044715x^3\right)\right)\right),$$
$$\mathrm{GELU}(x) \approx x\sigma(1.702x).$$

* Swish 和 GELU 都比传统分段线性函数更平滑, 目前在 Transformer 等模型中应用较多.

- **问题思考**:
  - 如何在一层网络中选择合适的激活函数以保障收敛速度?
  - 激活函数的非线性特性如何影响网络的拟合能力?

3.1 Sigmoid 的非零中心化问题

考虑模型 $y = f(w^\top \sigma(x))$, 其中 σ 为 Logistic 函数. 其关于权重 w 的导数为

$$\frac{\partial y}{\partial w} = \sigma(x) \cdot f'(w^\top \sigma(x)).$$

$\sigma(x)$ 是各个分量恒正的向量, $f'(w^\top \sigma(x))$ 则可正可负, 是一个标量. 梯度向量 $\frac{\partial y}{\partial w}$ 的各个分量会长期保持同号. 这样一来, 参数更新方向会被限制在少数固定象限内.

在二维参数空间 (w_1, w_2) 中, 更新方向通常只能落在第一或第三象限. 当真实最优方向不在这些方向上时, 梯度下降就容易走出明显的"之字形"路径, 从而降低收敛速度. 这也是为什么输出非零中心化的激活函数会给优化过程带来额外困难.

3.2 死亡 ReLU 问题

对 ReLU 激活函数

$$a = \mathrm{ReLU}(z), \quad z = \sum_{i=1}^{m} w_i a_i,$$

若某次迭代中 $z \le 0$, 则有 $\mathrm{ReLU}(z) = 0$, 且

$$\frac{\partial \mathrm{ReLU}}{\partial z} = 0.$$

因此, 对应权重的梯度满足

$$\frac{\partial a}{\partial w_i} = a_i \frac{\partial \mathrm{ReLU}}{\partial z} = 0.$$

这意味着该神经元一旦落入负半轴区域, 当前样本就无法再通过它产生梯度信号. 若这种情况持续发生, 则该神经元的参数长期得不到更新, 其输出也始终为 0, 最终形成所谓的"死亡 ReLU". 在深层网络中, 这种现象还会阻断更前面层的误差传递, 进一步影响训练效果.

4 反向传播算法

- 矩阵微积分: 介绍矩阵形式下的偏导数计算, 为反向传播奠定数学基础.
- 损失函数与梯度: 详细推导基于链式法则的梯度计算方法.

- 计算图与自动微分: 讨论如何利用计算图实现自动求导, 提升计算效率和代码的可维护性.
- **问题思考**:
  - 自动微分在大型网络中的实现细节和优化点在哪里?
  - 在反向传播中如何有效辨识与处理梯度消失或爆炸问题?

4.1 前向传播的矩阵形式

对于第 l 层, 前向传播可写为

$$\boldsymbol{a}^{(l)} = f_l(\boldsymbol{z}^{(l)}), \quad \boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}, \quad z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}.$$

这一形式把线性变换与非线性激活清晰地区分开来, 也为后续梯度推导提供了统一记号.

4.2 偏置 $b^{(l)}$ 的梯度

定义误差项

$$\delta^{(l)} = \frac{\partial L}{\partial \boldsymbol{z}^{(l)}},$$

由于

$$\frac{\partial \boldsymbol{z}^{(l)}}{\partial \boldsymbol{b}^{(l)}} = I_{M_l},$$

其中 I_{M_l} 为 $M_l \times M_l$ 的单位矩阵, 因此偏置的梯度为

$$\frac{\partial L}{\partial \boldsymbol{b}^{(l)}} = \frac{\partial \boldsymbol{z}^{(l)}}{\partial \boldsymbol{b}^{(l)}} \frac{\partial L}{\partial \boldsymbol{z}^{(l)}} = I_{M_l}\delta^{(l)} = \delta^{(l)}.$$

也就是说, 偏置项的梯度就是该层的误差项本身.

4.3 误差项的递推公式

当前层误差项可以由下一层误差项反向递推得到:

$$\delta^{(l)} = \frac{\partial L}{\partial \boldsymbol{z}^{(l)}} = \frac{\partial \boldsymbol{a}^{(l)}}{\partial \boldsymbol{z}^{(l)}} \frac{\partial \boldsymbol{z}^{(l+1)}}{\partial \boldsymbol{a}^{(l)}} \delta^{(l+1)}.$$

其中, 激活函数对输入的导数矩阵为

$$\left(\frac{\partial \boldsymbol{a}^{(l)}}{\partial \boldsymbol{z}^{(l)}}\right)_{ij} = f_l'\left(z_j^{(l)}\right)\delta_{ij},$$

因此可以写成对角矩阵形式

$$\frac{\partial \boldsymbol{a}^{(l)}}{\partial \boldsymbol{z}^{(l)}} = \operatorname{diag}\left(f_l'\left(z_1^{(l)}\right), \ldots, f_l'\left(z_{M_l}^{(l)}\right)\right) \in \mathbb{R}^{M_l \times M_l}.$$

同时,

$$\frac{\partial \boldsymbol{z}^{(l+1)}}{\partial \boldsymbol{a}^{(l)}} = \left(\boldsymbol{W}^{(l+1)}\right)^{\top}.$$

于是得到反向传播的核心递推关系

$$\delta^{(l)} = f_l'\left(\boldsymbol{z}^{(l)}\right) \odot \left(\left(\boldsymbol{W}^{(l+1)}\right)^\top \delta^{(l+1)}\right).$$

其中 $\odot$ 表示 Hadamard 乘积. 该公式说明: 当前层的误差由下一层误差经过权重转置映射回本层, 再与本层激活函数的导数逐元素相乘得到.

5 计算图与自动微分

- 基本思想:
  - 自动微分通过将复合函数分解为一系列基本运算, 并在此基础上系统地应用链式法则, 从而自动计算函数对各个变量的导数.
  - 这种分解方式可以表示为计算图. 图中的叶子节点对应输入变量或常量, 非叶子节点对应加法、乘法、激活函数等基本运算, 因而计算图给出了数学表达式的结构化表示.
- 前向模式与反向模式:
  - 设复合函数 $f(x; w, b)$ 可以分解为若干中间变量 $h_1, \ldots, h_6$, 则根据链式法则,

$$\frac{\partial f(x; w, b)}{\partial w} = \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w}.$$

  - 前向模式沿着计算图的正向传播方向递推导数信息, 更适合输入维数较小的情形.
  - 反向模式沿着计算图的反方向累积梯度, 当输出是标量时尤其高效. 若从输出到某个变量存在多条路径, 则该变量的总梯度等于所有路径贡献之和.
  - 神经网络中的反向传播算法, 本质上就是在计算图上执行反向模式自动微分.
- 在前馈神经网络中的作用:
  - 在参数学习问题中, 损失函数通常可看作 $L(\theta) : \mathbb{R}^N \to \mathbb{R}$, 即输入为全部参数、输出为单个标量损失. 对这类问题, 反向模式通常比逐个参数进行前向求导更高效.
  - 前馈神经网络的训练过程通常包括三步: 先前向计算各层的净输入与激活值, 再反向计算各层参数的梯度, 最后利用优化算法更新参数.
- 静态计算图与动态计算图:
  - 静态计算图在程序运行前完成图结构的构建, 随后再执行计算. 这种方式便于做图级优化和并行调度, 但模型结构的灵活性相对较弱. TensorFlow 2.0 之前的默认范式主要属于这一类.
  - 动态计算图在程序执行过程中即时构建, 更便于调试, 也更适合处理不同输入对应不同网络结构的情形. PyTorch 是典型代表.
  - 二者各有取舍: 静态图通常更容易做全局优化, 动态图通常具有更高的表达灵活性.
- **问题思考**:
  - 当网络中包含分支结构、共享参数或循环结构时, 计算图上的梯度累积会出现哪些额外问题?
  - 在实际深度学习框架中, 动态计算图的灵活性与静态计算图的执行效率应如何权衡?

6 优化问题

- 优化问题的非凸性:
  - 即使是非常简单的多层网络, 其关于参数的目标函数通常也是非凸的. 例如对一个 1-1-1 结构且采用 Sigmoid 激活的网络,

$$y = \sigma\left(w_2 \sigma\left(w_1 x\right)\right),$$

    输出关于参数 w_1, w_2 的关系已经是嵌套非线性的, 因而训练目标一般不再具有凸优化中的良好性质.
  - 在这类问题中, 基于梯度的方法通常只能保证收敛到某个驻点, 该驻点可能是局部极小值, 也可能是鞍点. 尤其在高维参数空间中, 鞍点往往比严格的局部极值点更常见.
  - 这意味着神经网络训练不仅要面对搜索空间巨大这一困难, 还要处理目标函数曲面复杂、不同方向曲率差异显著等问题.
- 梯度消失现象:
  - 在深层网络中, 反向传播需要不断连乘各层的导数. 若这些导数的绝对值长期小于 1, 则梯度会随层数增加而指数衰减, 从而使靠近输入端的参数难以得到有效更新.
  - Sigmoid 和 Tanh 等饱和型激活函数在输入较大或较小时导数接近于 0, 因而更容易引发梯度消失. 这也是深层网络较少直接使用这类激活函数作为隐藏层主激活的原因之一.
  - ReLU 在正半轴上的导数恒为 1, 因而在一定程度上缓解了梯度消失问题, 但它并不能从根本上消除所有训练困难. 在实际中, 还常配合合理初始化、归一化方法和残差连接等技术共同改善优化过程.
- 训练中的实际困难与需求:
  - 神经网络通常包含大量参数, 训练过程对计算资源和存储资源的需求较高; 当样本规模不足时, 还容易出现泛化性能下降的问题.
  - 模型参数数量多、层间耦合强, 使得训练行为分析和参数可解释性都相对困难.
  - 因此, 成功训练深度神经网络往往依赖三个方面的条件: 足够的数据、足够的计算资源, 以及收敛速度和稳定性都较好的优化算法.
- **问题思考**:
  - 非凸优化问题在实际应用中如何选择合适的优化算法?
  - 面对梯度消失现象, 有哪些前沿的方法可以有效改善网络训练?

7 小结

本讲介绍了前馈神经网络的基本概念、结构与训练方法:

- 前馈神经网络通过多层非线性变换增强模型的表达能力, 其核心在于激活函数的引入.
- 激活函数的选择直接影响网络的收敛性和拟合能力. ReLU 因其计算效率高而被广泛应用, 但需注意其潜在的" 死亡" 问题.

- 反向传播算法利用链式法则高效计算梯度，是现代深度学习的基础. 自动微分框架 (如 PyTorch) 大幅简化了其实现.
- 神经网络的优化问题本质上是非凸的，梯度消失与梯度爆炸是训练深层网络的主要挑战. 合理的网络初始化、归一化技术和优化算法选择对训练成功至关重要.

8 参考资料

- Li, H. Y. (2025). 机器学习 (国立台湾大学公开课). Retrieved from https://www.bilibili.com/video/BV1Wv411h7kN/
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.